Now that we know two concrete data structures to implement the *ADT* `StringContainer`, we can confidently look at applications where they make a difference.

As usual with data structures, the difference is not about new things that couldn't be done without tries, but rather about doing some operations better or faster than with other DSs.

This is particularly true for tries, as they were specifically designed to improve the running time of string-based queries. As one of the main uses for tries is to implement text-based dictionaries, the touchstone will often be hast tables.

6.4.1 Spell Checker

Time to go back to our main example!

We have seen in chapter 4 that Bloom filters where used for the first versions of spell checkers, but after a while they were replaced with more efficient alternatives, like tries.

The first step to build a spell-checker is, obviously, inserting all the keys from our dictionary (here meant as “English dictionary”, not like the data structure!) in a trie.

Then, using the trie for spell-check when the feedback we want is just highlighting typos is simple: we just need to perform a search, and if it’s a miss, we have a typo.

But suppose, instead, that we would like to also provide suggestions about how we could correct the typo: how can we do that with a trie?